

# PRACTICAL - 09

---

**Name:** Hansika Reddy Gunapati

**Rollno:** 2520030237

**Section:** 8

---

## PART 1: FILE COPY USING LINUX SYSTEM CALLS

### Title

**File Copy Using Low-Level Linux I/O and Performance Comparison**

### Aim

To implement a file copy utility using `open()`, `read()`, `write()`, `lseek()`, and `close()`, and compare its performance with `fopen()`, `fread()`, and `fwrite()`.

### Objectives

- To understand Linux file handling and file descriptors.
- To use low-level file I/O system calls.
- To copy data from one file to another.
- To compare system calls with standard library functions.

### Theory

Linux provides system calls for file input and output. `open()` returns a file descriptor, `read()` reads data into a buffer, `write()` writes data to a file, `lseek()` changes the file position, and `close()` closes the file descriptor. Standard library functions such as `fopen()`, `fread()`, `fwrite()`, and `fclose()` provide higher-level file handling.

### System Calls / Functions

| Function             | Purpose              |
|----------------------|----------------------|
| <code>open()</code>  | Opens/creates a file |
| <code>read()</code>  | Reads data           |
| <code>write()</code> | Writes data          |

|          |                          |
|----------|--------------------------|
| lseek()  | Changes file position    |
| close()  | Closes file              |
| fopen()  | Opens file using library |
| fread()  | Reads using library      |
| fwrite() | Writes using library     |

## Algorithm

1. Start the program.
2. Open the source file using open().
3. Create the destination file using open().
4. Read data using read().
5. Write data using write().
6. Repeat until the complete file is copied.
7. Use lseek() to change the file position.
8. Close both files using close().
9. Repeat the process using fopen(), fread(), and fwrite().
10. Compare their execution times.
11. Stop.

## C Program

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytes;

    source = open("source.txt", O_RDONLY);

    if (source == -1)
    {
        perror("Error opening source");
        return 1;
    }

    destination = open("destination.txt",
        O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination == -1)
    {
```

```
    perror("Error opening destination");
    close(source);
    return 1;
}

while ((bytes = read(source, buffer, sizeof(buffer))) > 0)
    write(destination, buffer, bytes);

lseek(source, 0, SEEK_SET);

close(source);
close(destination);

printf("File copied successfully.\n");

return 0;
}
OUTPUT -
```

```

teja@Hasini:~$ ls
Experiment-2      pipeline
abc.c            pipeline.c
filecopy.c       producer_consumer
filecopy.c.save  producer_consumer.c
filecopy.c.save.1 producer_consumer.c.save
fork             producer_consumer.c.save.1
fork.c           program.c
hello            question
hello.c          question.c
hello.txt        question7
input.txt        question7.c
linux            question9
merged.txt       question9.c
only_err.txt     quotes
only_out.txt     quotes.c
ordered.txt      readonly.txt
os_task          redirection
os_task.c        redirection.c
output.txt       redirection_pipeline
parser           redirection_pipeline.c
parser.c         result.txt
path_resolver    sample.py
path_resolver.c
teja@Hasini:~$ nano source.txt
teja@Hasini:~$ ls
Experiment-2      pipeline
abc.c            pipeline.c
filecopy.c       producer_consumer
filecopy.c.save  producer_consumer.c
filecopy.c.save.1 producer_consumer.c.save
fork             producer_consumer.c.save.1

```

## Performance Comparison

Use:

time ./lowlevel\_copy

and:

time ./standard\_copy

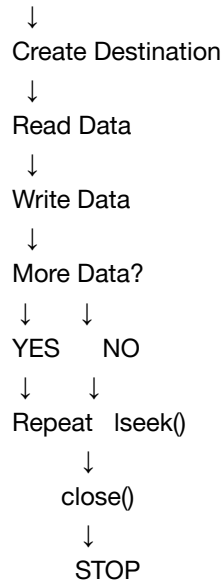
The actual execution time depends on file size, buffer size, storage device, and system load.

## Flow Diagram

START

↓

Open Source File



### Observation

The file was successfully copied using low-level system calls. The same operation can be performed using standard library functions, and their execution times can be compared.

### Result

The file copy utility was successfully implemented using Linux system calls and its performance can be compared with standard library file operations.

## PART 2: I/O REDIRECTION USING dup2()

### Title

**Standard Input and Output Redirection Using File Descriptors and dup2()**

### Aim

To develop a C program that redirects standard input and output using file descriptors and dup2(), demonstrating how shells implement I/O redirection.

### Objectives

- To understand standard input and output.
- To understand file descriptors.
- To use dup2() for redirection.
- To redirect input from a file and output to another file.

## Theory

Linux provides three standard file descriptors:

0 → Standard Input

1 → Standard Output

2 → Standard Error

The dup2() system call duplicates one file descriptor to another.

```
dup2(input_fd, STDIN_FILENO);
```

```
dup2(output_fd, STDOUT_FILENO);
```

After redirection, input comes from the input file and output is written to the output file.

For example:

```
cat < input.txt > output.txt
```

The shell opens both files and uses file descriptor duplication before executing the command.

## Algorithm

1. Start the program.
2. Open input.txt.
3. Open/create output.txt.
4. Use dup2() to redirect standard input.
5. Use dup2() to redirect standard output.
6. Read data from standard input.
7. Write data to standard output.
8. Close the files.
9. Stop.

## C Program

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    int input_fd, output_fd;
```

```
    char buffer[1024];
```

```
    ssize_t bytes;
```

```
    input_fd = open("input.txt", O_RDONLY);
```

```
    output_fd = open("output.txt",  
                    O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
    if (input_fd == -1 || output_fd == -1)
```

```
    {
```

```
        perror("File error");
```

```
        return 1;
```

```
}

dup2(input_fd, STDIN_FILENO);
dup2(output_fd, STDOUT_FILENO);

while ((bytes = read(STDIN_FILENO, buffer, sizeof(buffer))) > 0)
    write(STDOUT_FILENO, buffer, bytes);

close(input_fd);
close(output_fd);

return 0;
}
```

OUTPUT -

```

teja@Hasini:~$ ls
Experiment-2      path_resolver
abc.c             path_resolver.c
destination.txt   pipeline
dup2copy          pipeline.c
dup2copy.c        producer_consumer
filecopy           producer_consumer.c
filecopy.c         producer_consumer.c.save
filecopy.c.save    producer_consumer.c.save.1
filecopy.c.save.1 program.c
fork              question
fork.c            question.c
hello             question7
hello.c           question7.c
hello.txt         question9
input.txt         question9.c
linux            quotes
merged.txt        quotes.c
only_err.txt      readonly.txt
only_out.txt      redirection
ordered.txt       redirection.c
os_task           redirection_pipeline
os_task.c         redirection_pipeline.c
output.txt        result.txt
parser            sample.py
parser.c          source.txt
teja@Hasini:~$ cat output.txt

Hello, this is the input file.
This content will be copied to the output file.
delta
apple
cherry
banana
apple
cherry
apple

```

## Compilation

```
gcc redirect.c -o redirect
```

## Execution

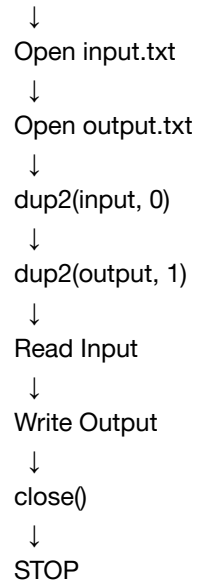
```
./redirect
```

Check the output:

```
cat output.txt
```

## Flow Diagram

START



## Observation

The input file was successfully connected to standard input and the output file was connected to standard output using dup2(). This demonstrates the basic mechanism used by shells for I/O redirection.

## Result

Standard input and output were successfully redirected using dup2(), demonstrating how Linux shells implement file-based I/O redirection.